

Pontificia universidad católica del Ecuador

PUCE VIRTUAL



INGENIERIA EN CIBERSEGURIDAD Y GESTION DE TI

QUINTO NIVEL

INTELIGENCIA ARTIFICIAL

DOCENTE:

Rafael Melgarejo Heredia

Tarea - Semana 2 Clasificación inteligente de eventos de seguridad mediante Machine Learning

AUTOR:

Alejandro Carriel

Segundo periodo ordinario

INFORME TÉCNICO

Clasificación inteligente de eventos de seguridad mediante Machine Learning

CIC-IDS2017 (versión académica curada) · Aprendizaje supervisado binario

1. Introducción y objetivo

Esta actividad construye un primer sistema inteligente para clasificar eventos de seguridad a partir de flujos de red. El problema es aprendizaje supervisado: cada fila resume un flujo con variables numéricas estilo CICFlowMeter y una etiqueta indica si es benigno o malicioso. El agente percibe esas features, decide la clase y actúa alertando o no.

Se usa la versión académica curada de CIC-IDS2017, con train y test ya definidos, tres objetivos (label_binary, attack_family, label_multiclass) y source_file como trazabilidad. Esta semana solo predice label_binary: ¿hay que alertar o no?

El protocolo sigue la consigna: source_file no entrena; las tres etiquetas no son predictores; el test no se usa para entrenar, elegir variables ni tunear hiperparámetros; el escalado se estima solo en train con un Pipeline; la exactitud no es la métrica única, por el desbalance residual y porque en ciberseguridad no todos los errores cuestan lo mismo.

2. Marco conceptual

2.1 Aprendizaje supervisado y agentes en un IDS

En aprendizaje supervisado el algoritmo estima una función $f(x) \approx y$ a partir de ejemplos etiquetados. Aquí x es el vector de 70 features de flujo e y es label_binary (0 = benigno, 1 = ataque). Un árbol, un bosque y una SVM son tres hipótesis distintas sobre f . En términos de agente inteligente, la percepción son las estadísticas del flujo, la decisión es la clase predicha y la actuación es el disparo de una alerta hacia el analista.

2.2 Árbol de decisión, Random Forest y SVM

El árbol de decisión particiona el espacio con umbrales univariados que reducen la impureza (índice de Gini). El resultado es un conjunto de reglas auditables. Random Forest construye muchos árboles sobre submuestras de filas y de variables; el voto reduce la varianza respecto a un árbol único. La máquina de soporte vectorial busca el hiperplano de máximo margen. Con kernel RBF la frontera puede ser no lineal, pero el método es sensible a la escala: variables como flow_duration (órdenes de 10^8) aplastarían a un contador de flags si no se estandarizan. Por eso SVM se envuelve en un Pipeline con StandardScaler ajustado exclusivamente con train.

3. Datos, predictores y auditoría

3.1 Carga y respeto de la partición

Se cargaron por separado CICIDS2017_train.csv y CICIDS2017_test.csv. No se concatenaron ni se volvió a ejecutar un train_test_split sobre el conjunto mezclado. Train contiene 24 000 flujos y test 6 000; ambos tienen 74 columnas. Las cuatro últimas son label_multiclass, attack_family, label_binary y source_file.

```
train = pd.read_csv("data/CICIDS2017_train.csv", low_memory=False) test = pd.read_csv("data/CICIDS2017_test.csv", low_memory=False)
print(train.shape, test.shape) # (24000, 74) (6000, 74)
```

3.2 Distribución de label_binary

La etiqueta binaria toma solo los valores 0 y 1. En entrenamiento hay 12 736 flujos benignos (53,07 %) y 11 264 ataques (46,93 %). En prueba las cifras son 3 184 y 2 816, con la misma proporción 53,07 % / 46,93 %. El desbalance es residual, no extremo. Aun así, un clasificador trivial que predijera siempre “benigno” alcanzaría aproximadamente 53 % de exactitud y no detectaría ningún incidente. Esa es la razón de evaluar precision, recall, F1 y ROC-AUC de la clase ataque, además del recuento de falsos negativos.

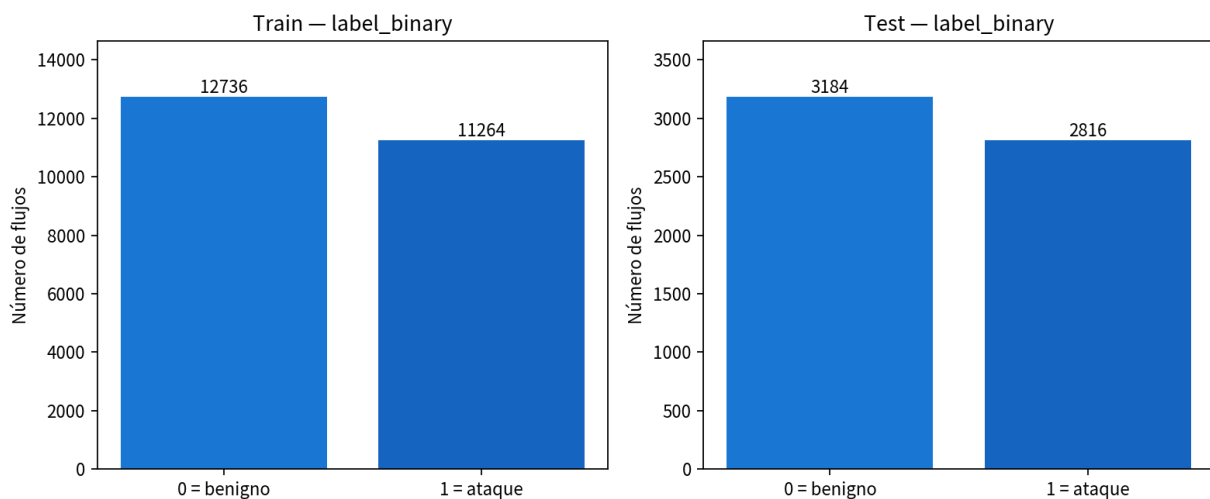


Figura 1. Distribución de label_binary en las particiones oficiales de entrenamiento y prueba.

3.3 Exclusión de columnas y predictores

Del conjunto de entrada se excluyeron label_binary (objetivo), attack_family, label_multiclass y source_file. Las dos etiquetas adicionales no pueden usarse como predictores porque contienen información que el modelo debe inferir. source_file identifica el CSV diario original y permitiría aprender el archivo en lugar de la firma de red. Quedaron 70 variables numéricas: puerto destino, duración, conteos y longitudes de paquetes, tasas de bytes y paquetes, tiempos entre llegadas, flags TCP, tamaños medios, ventanas iniciales y estadísticas active/idle.

Las matrices resultantes son X_{train} ($24\,000 \times 70$), y_{train} ($24\,000$), X_{test} ($6\,000 \times 70$) e y_{test} ($6\,000$). Ninguna columna prohibida permanece en X .

3.4 Estadísticos y calidad

Las descriptivas se calcularon solo sobre entrenamiento. Se observa una heterogeneidad de escala importante: `destination_port` llega a 65 501, `flow_duration` a $1,20 \times 10^8$ y `total_length_of_bwd_packets` a $4,75 \times 10^8$, mientras que los flags viven en enteros pequeños. Esa diferencia justifica el escalado de SVM y, al mismo tiempo, explica por qué los árboles no lo necesitan: sus cortes son invariantes a transformaciones monótonas de una feature.

La auditoría de calidad no encontró valores nulos ni infinitos en predictores de train ni de test, ni filas duplicadas en los CSV originales (0 / 0 en los tres chequeos). El conjunto académico ya viene curado; no se aplicó una segunda limpieza, de acuerdo con la consigna.

4. Modelos implementados

Se implementaron los tres algoritmos previstos en la rúbrica, con `scikit-learn`. Los hiperparámetros se fijaron a priori, sin búsqueda sobre el test. Los tres usan `class_weight='balanced'` para no ignorar la clase ataque. `random_state=42` garantiza reproducibilidad.

- Árbol de decisión: `max_depth=12` y `min_samples_leaf=5`, para limitar el sobreajuste y conservar reglas interpretables.
- Random Forest: 100 árboles, `max_depth=18`, `max_features='sqrt'`. No requiere escalado.
- SVM RBF dentro de Pipeline(`StandardScaler` → `SVC`), con `C=1.0`, `gamma='scale'` y `probability=True` para habilitar la curva ROC.

El Pipeline es el mecanismo que cumple la regla de escalado. Al llamar `fit` sobre X_{train} , `StandardScaler` estima media y desviación solo en entrenamiento y, a continuación, `SVC` aprende sobre los datos ya estandarizados. Al llamar `predict` sobre X_{test} se reutilizan esos mismos parámetros. No se hace `scaler.fit(X_test)`.

```
svm_pipe = Pipeline([ ("escalado", StandardScaler()), ("svm", SVC(kernel="rbf", C=1.0, class_weight="balanced",
probability=True, random_state=42)), ]) svm_pipe.fit(X_train, y_train)
```

5. Evaluación en el conjunto de prueba

Todas las cifras de esta sección se obtuvieron exclusivamente sobre los 6 000 flujos de test. La clase positiva es el ataque (`pos_label=1`). En la matriz de confusión, TN son benignos correctos, FP son benignos alertados, FN son ataques etiquetados como benignos y TP son ataques detectados. `Recall de ataque = TP/(TP+FN)` mide la fracción de incidentes que el IDS no deja pasar. `Precision = TP/(TP+FP)` mide la calidad de las alertas. F1 combina ambas. ROC-AUC resume la capacidad de ordenar flujos por score de ataque.

Modelo	Acc.	Prec.	Recall	F1	AUC	FN	FP
Árbol de decisión	0.9940	0.9926	0.9947	0.9936	0.9981	15	21
Random Forest	0.9948	0.9975	0.9915	0.9945	0.9998	24	7
SVM + Pipeline	0.9410	0.9184	0.9595	0.9385	0.9880	114	240

Tabla 1. Métricas en test. Precision, recall y F1 corresponden a la clase ataque.

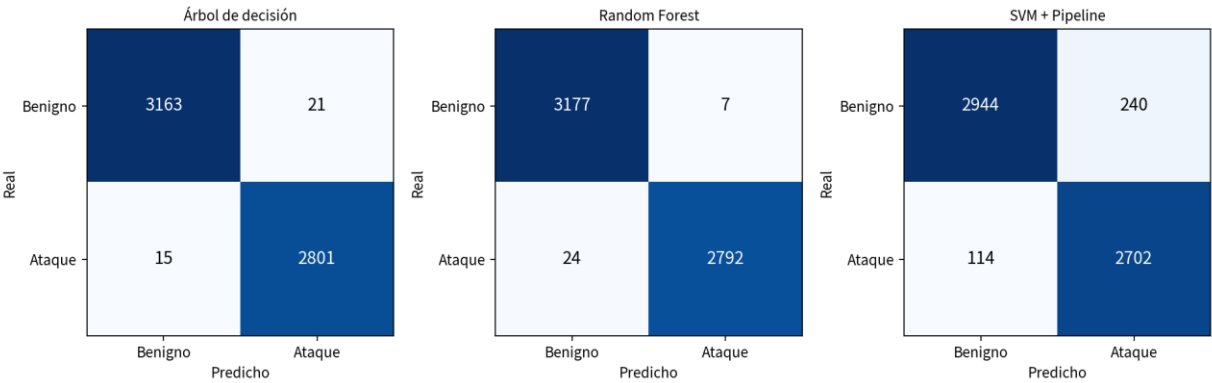


Figura 2. Matrices de confusión en test. Filas = clase real; columnas = clase predicha.

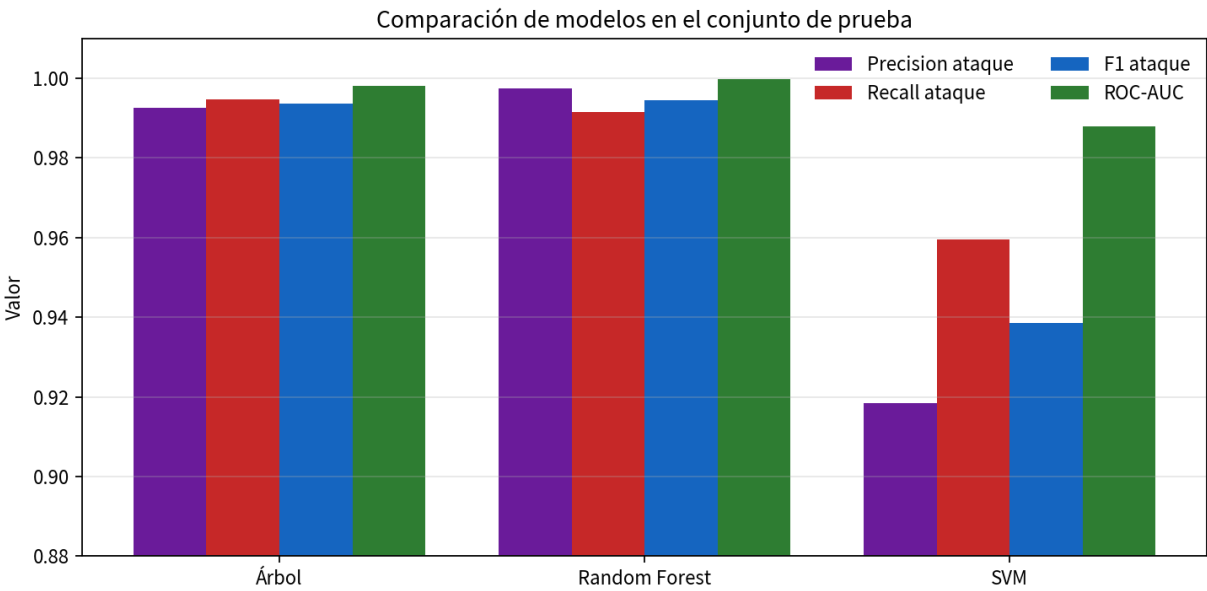


Figura 3. Comparación de precision, recall, F1 de ataque y ROC-AUC.

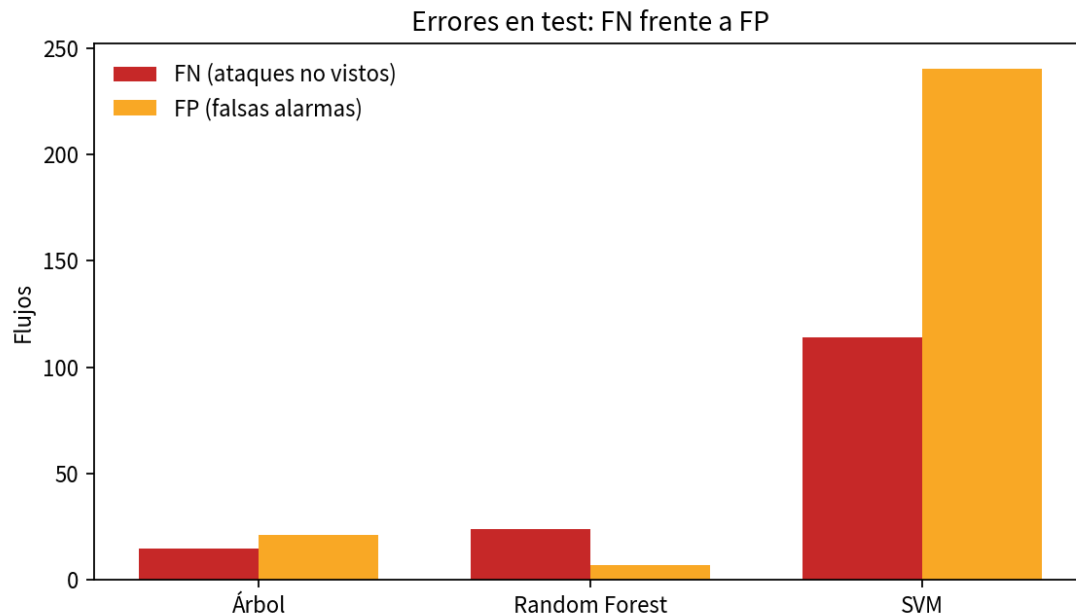


Figura 4. Errores en test: falsos negativos frente a falsos positivos.

El árbol y Random Forest se sitúan ambos por encima de 99 % en accuracy, F1 y AUC; la diferencia operativa está en el tipo de error. SVM, pese a un Pipeline correcto y un AUC de 0,988, produce 114 FN y 240 FP, claramente por debajo de los ensembles de árboles en este conjunto tabular de flujos.

6. Falsos negativos

Los falsos negativos son ataques clasificados como tráfico benigno. En un SOC se traducen en dwell time: el adversario opera sin alerta. El árbol registró 15 FN de 2 816 ataques de test (0,53 %). Random Forest dejó pasar 24 (0,85 %) y SVM 114 (4,05 %).

Cruzando los 15 FN del árbol con `attack_family` —solo como diagnóstico, nunca como predictor— la distribución es: DoS 7, Infiltration 3, Botnet 2, BruteForce 2 y WebAttack 1. DoS concentra más FN en valor absoluto porque es una familia frecuente; Infiltration, Botnet y WebAttack pesan más en términos relativos, porque su firma se parece más a tráfico legítimo o porque tienen menos ejemplos para aprender.

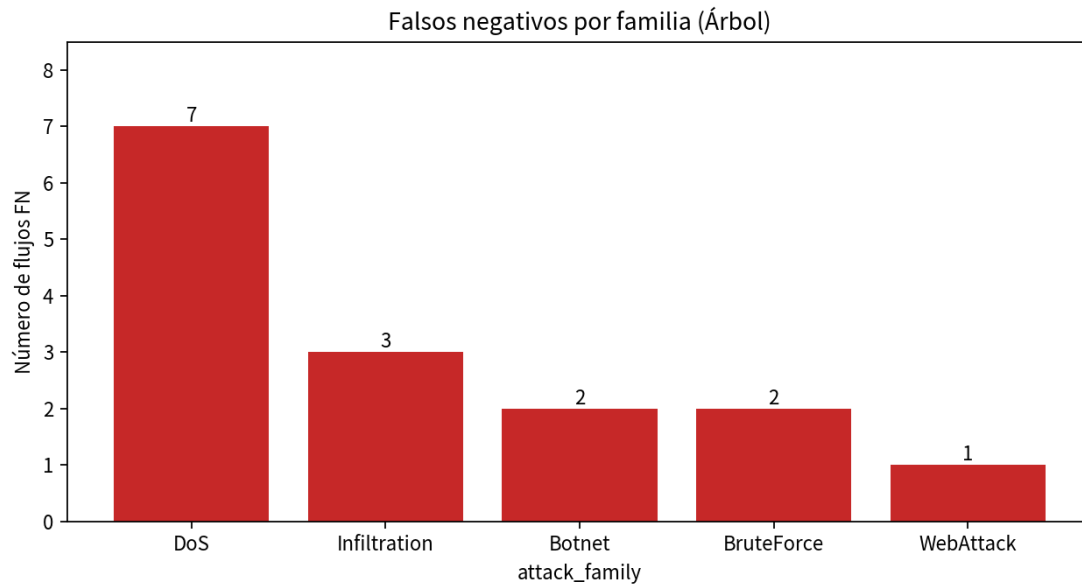


Figura 5. Falsos negativos del árbol de decisión desglosados por `attack_family`.

En operación, el umbral de decisión podría bajarse usando `predict_proba` para ganar recall a costa de más FP, según la capacidad de triaje del equipo. Ese ajuste no se hizo sobre el test durante el modelado, para no violar el protocolo.

7. Selección del modelo

El modelo más adecuado desde una perspectiva de ciberseguridad es el árbol de decisión. La justificación no es la exactitud —Random Forest incluso la supera por ocho diezmilésimas—, sino el coste asimétrico de los errores. Un falso negativo deja pasar un incidente; un falso positivo genera una alerta extra. La consigna pide analizar especialmente los FN, y el árbol es el que menos ataques clasifica como benignos (15 frente a 24 y 114).

Random Forest permanece como alternativa operativa sólida: solo 7 FP y ROC-AUC 0,9998, útil si la prioridad del SOC fuera no saturar al analista. SVM demuestra que el Pipeline y el escalado se implementaron de forma correcta, pero no es competitivo en este régimen. Ningún modelo se eligió mirando únicamente accuracy.

8. Limitaciones y trabajo futuro

CIC-IDS2017, aun curado, sale de un entorno controlado: un recall alto aquí no garantiza otra red. La tarea binaria no nombra el ataque; las otras etiquetas quedan para una segunda etapa, sin usarlas como features. No hubo GridSearch con CV interna en train; sería el siguiente paso, sin tocar el test. A futuro: umbral por coste FN/FP y un baseline que siempre predice benigno.

9. Conclusiones

Se implementó un clasificador binario sobre la partición académica de CIC-IDS2017, con 70 predictores de flujo y exclusión explícita de etiquetas y de source_file. La auditoría no halló nullos, infinitos ni duplicados. Se entrenaron un árbol de decisión, un Random Forest y una SVM con Pipeline de escalado, y se evaluaron solo en test mediante matriz de confusión, precision, recall, F1 y ROC-AUC. El árbol de decisión se selecciona como detector principal porque minimiza los falsos negativos (15 de 2 816 ataques), que son el error de mayor impacto en un IDS. Los FN remanentes se concentran en DoS y, en proporción, en familias menos representadas. El principio que deja la práctica es que un agente de seguridad no se mide solo por cuántos flujos acierta, sino por cuántos incidentes es capaz de no dejar pasar.

Referencias

Sharafaldin, I., Lashkari, A. H. y Ghorbani, A. A. (2018). Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. ICISSP.

Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.

Cortes, C. y Vapnik, V. (1995). Support-vector networks. Machine Learning, 20, 273–297.

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Canadian Institute for Cybersecurity. CIC-IDS2017. University of New Brunswick.